Name: Sathwik P Bhat
Reg No: 230905012
Roll No: 4

Lab Exercises:

1. Write a program in CUDA to perform matrix multiplication using 2D Grid and 2D Block.

CODE

```
#include<stdio.h>
#include<stdlib.h>
#include<cuda.h>

#define N 2

__global__ void multiply(int* a, int* b, int* res){
   int x = blockIdx.y * blockDim.y + threadIdx.y;
   int y = blockIdx.x * blockDim.x + threadIdx.x;

   int sum = 0;

   for(int k = 0; k < N; k++){
      sum += a[x*N+k] * b[k*N + y];
   }
   res[x*N+y] = sum;
}

int main(){

   int size = N*N*sizeof(int);

   int* a = (int*)malloc(size);
   int* b = (int*)malloc(size);
   int* res = (int*)malloc(size);
   int *d_a, *d_b, *d_res;

   printf("Enter matrix1: \n");
   for(int i = 0; i < N*N; i++){
     scanf("%d", &a[i]);
   }

   printf("Enter matrix2: \n");
   for(int i = 0; i < N*N; i++){
     scanf("%d", &b[i]);
   }

   cudaMalloc((void**)&d_a, size);
   cudaMalloc((void**)&d_b, size);
   cudaMalloc((void**)&d_res, size);

   cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
   cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);
```

```
    dim3 grid(1,1);
    dim3 block(2,2,1);

    multiply<<<grid,block>>>(d_a, d_b, d_res);
    cudaMemcpy(res, d_res, size, cudaMemcpyDeviceToHost);

    printf("Resultant matrix: \n");
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t", res[i*N+j]);
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT

```
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ nvcc q1.cu -o q1
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ ./q1
Enter matrix1:
1 2 3 4
Enter matrix2:
5 6 7 8
Resultant matrix:
19      22
43      50
```

2. Write a program in CUDA to improve the performance of 1D parallel convolution using constant Memory.

CODE

```
#include <stdio.h>
#include <cuda.h>

#define N 8
#define MASK_WIDTH 3

__constant__ int d_mask[MASK_WIDTH];

__global__ void convolution_1D(int *input, int *output, int n){
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if(i < n){
        int result = 0;

        int radius = MASK_WIDTH / 2;

        for(int j = 0; j < MASK_WIDTH; j++){
```

```
            int idx = i - radius + j;

            if(idx >= 0 && idx < n){
                result += input[idx] * d_mask[j];
            }
        }

        output[i] = result;
    }
}

int main(){

    int h_input[N] = {1,2,3,4,5,6,7,8};
    int h_mask[MASK_WIDTH] = {1,0,-1};
    int h_output[N];

    int *d_input, *d_output;

    int size = N * sizeof(int);


    cudaMalloc((void**)&d_input, size);
    cudaMalloc((void**)&d_output, size);

    cudaMemcpy(d_input, h_input, size, cudaMemcpyHostToDevice);

    cudaMemcpyToSymbol(d_mask, h_mask, MASK_WIDTH * sizeof(int));


    convolution_1D<<<ceil(N/256.0), 256>>>(d_input, d_output, N);

    cudaMemcpy(h_output, d_output, size, cudaMemcpyDeviceToHost);

    printf("Output:\n");
    for(int i = 0; i < N; i++){
        printf("%d ", h_output[i]);
    }

    printf("\n");

    cudaFree(d_input);
    cudaFree(d_output);

    return 0;
}
```

OUTPUT

```
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ nvcc q2.cu -o q2
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ ./q2
Output:
-2 -2 -2 -2 -2 -2 -2 7
```

3. Write a program in CUDA to perform tiled 1D convolution operation on the input array N of size *width* using the mask array, M of size *mask_width,* to produce the resultant array P of size *width*

CODE

```
#include <stdio.h>
#include <cuda.h>

#define WIDTH 16
#define MASK_WIDTH 5
#define TILE_SIZE 8

__constant__ int M[MASK_WIDTH];

__global__ void tiled_convolution_1D(int *N, int *P, int width) {

    __shared__ int N_s[TILE_SIZE + MASK_WIDTH - 1]; // shared memory with halo

    int tx = threadIdx.x;
    int bx = blockIdx.x;

    int radius = MASK_WIDTH / 2;


    int g_index = bx * TILE_SIZE + tx;

    int s_index = tx + radius;

    if (g_index < width)
        N_s[s_index] = N[g_index];
    else
        N_s[s_index] = 0;

    if (tx < radius) {
        int left_index = g_index - radius;
        if (left_index >= 0)
            N_s[tx] = N[left_index];
        else
            N_s[tx] = 0;
    }

    if (tx < radius) {
        int right_index = g_index + TILE_SIZE;
        if (right_index < width)
            N_s[s_index + TILE_SIZE] = N[right_index];
        else
```

```
        N_s[s_index + TILE_SIZE] = 0;
    }

    __syncthreads();

    if (g_index < width) {
        int result = 0;
        for (int j = 0; j < MASK_WIDTH; j++) {
            result += N_s[tx + j] * M[j];
        }
        P[g_index] = result;
    }
}

int main() {

    int h_N[WIDTH] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};
    int h_M[MASK_WIDTH] = {1,0,-1,0,1};
    int h_P[WIDTH];

    int *d_N, *d_P;

    int size = WIDTH * sizeof(int);

    cudaMalloc((void**)&d_N, size);
    cudaMalloc((void**)&d_P, size);

    cudaMemcpy(d_N, h_N, size, cudaMemcpyHostToDevice);

    cudaMemcpyToSymbol(M, h_M, MASK_WIDTH * sizeof(int));

    int gridSize = (WIDTH + TILE_SIZE - 1) / TILE_SIZE;

    tiled_convolution_1D<<<gridSize, TILE_SIZE>>>(d_N, d_P, WIDTH);

    cudaMemcpy(h_P, d_P, size, cudaMemcpyDeviceToHost);

    printf("Output:\n");
    for (int i = 0; i < WIDTH; i++) {
        printf("%d ", h_P[i]);
    }
    printf("\n");

    cudaFree(d_N);
    cudaFree(d_P);

    return 0;
}
```

OUTPUT

```
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ nvcc q3.cu -o q3
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ ./q3
Output:
2 2 3 4 5 6 7 8 9 10 11 12 13 14 -2 -2
```

Additional Exercise:

1. Write a program in CUDA which displays a shopping mall item menu with its price. The N number of friends are allowed to purchase as many items they want. Calculate the total purchase done by N friends.

CODE

```
#include <stdio.h>
#define N 4

__global__ void totalPurchase(int *prices, int *quantities, int *total) {
int i = threadIdx.x;
total[i] = prices[i] * quantities[i];
}

int main() {
        int prices[N], quantities[N], total[N], sum = 0;
        printf("Enter item prices:\n");
        for(int i=0;i<N;i++) scanf("%d",&prices[i]);
        printf("Enter quantities bought by friend:\n");
        for(int i=0;i<N;i++) scanf("%d",&quantities[i]);
        int *d_p, *d_q, *d_t;
        cudaMalloc(&d_p, N*sizeof(int));
        cudaMalloc(&d_q, N*sizeof(int));
        cudaMalloc(&d_t, N*sizeof(int));
        cudaMemcpy(d_p, prices, N*sizeof(int), cudaMemcpyHostToDevice);
        cudaMemcpy(d_q, quantities, N*sizeof(int), cudaMemcpyHostToDevice);
        totalPurchase<<<1, N>>>(d_p, d_q, d_t);
        cudaMemcpy(total, d_t, N*sizeof(int), cudaMemcpyDeviceToHost);
        printf("Individual totals:\n");
        for(int i=0;i<N;i++) {
        printf("%d ", total[i]);
        sum += total[i];
}
printf("\nTotal purchase = %d\n", sum);
return 0;
}
```

OUTPUT

```
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ nvcc q4.cu -o q4
6CSED1@debian:~/Documents/Sathwik230905012/lab10$ ./q4
Enter item prices:
10 20
30
40
Enter quantities bought by friend:
1
2
3
1
Individual totals:
10 40 90 40
Total purchase = 180
```